

APOSTILA COMPLETA

IntelliJ & Lógica Condicional

Da IDE ao domínio das condições e operadores booleanos

if-then • AND && / OR || • Ternário • Operadores • Precedência

7 seções • Lógica booleana • Operadores e Ternário

Complete Java Masterclass

Sumário

01	Hello World no IntelliJ	IDE, modificadores de acesso, classes e método main
02	Instrução if-then	Estrutura if, blocos de código, boas práticas
03	= vs == e o Operador NOT !	Atribuição vs igualdade, complemento lógico
04	AND && e OR 	Combinando condições, tabelas-verdade, short-circuit
05	Precedência de Operadores	Ordem de avaliação e parênteses
06	Operador Ternário ?:	Sintaxe, usos corretos e anti-padrões
07	Boas Práticas e Erros Comuns	Convenções, atalhos IntelliJ, resumo

CAPÍTULO 01

Hello World no IntelliJ

Configurando a IDE, entendendo modificadores e o ponto de entrada da JVM

O que é o IntelliJ IDEA?

O IntelliJ IDEA é a IDE (Integrated Development Environment) mais popular para Java, desenvolvida pela JetBrains. Está disponível em versão Community (gratuita) e Ultimate. Diferente do JShell, o IntelliJ organiza seu código em projetos com múltiplos arquivos, detecta erros de compilação enquanto você digita e oferece autocomplete inteligente, refatoração automática e um debugger poderoso.

```
// Arquivo: PrimeiraClasse.java

public class PrimeiraClasse {

    // Método main: ponto de entrada de todo programa Java

    public static void main(String[] args) {

        System.out.println("Hello World");

    }

}
```

Modificadores de Acesso

Modificadores de acesso controlam a visibilidade de classes, métodos e variáveis. Java tem quatro: **public** (acessível de qualquer lugar), **protected** (classe, pacote e subclasses), package-private (sem modificador — apenas no mesmo pacote) e **private** (apenas dentro da própria classe). A classe principal e o método main devem ser **public** para que a JVM os encontre.

■ **Dica:** Atalhos essenciais: psvm + Tab gera public static void main(String[] args) {}; sout + Tab gera System.out.println(). Ctrl+Space ativa autocomplete; Alt+Enter sugere correções automáticas.

CAPÍTULO 02

Instrução if-then

Controlando o fluxo do programa com condições verdadeiro/falso

O que é Lógica Condicional?

A lógica condicional permite que seu programa tome decisões: executar código apenas quando uma condição for satisfeita. Sem ela, todo programa executaria sempre os mesmos passos na mesma ordem. O if é a estrutura mais básica de controle de fluxo — presente em praticamente toda linguagem de programação moderna.

```
boolean isAlien = false;

// ■ FORMA RECOMENDADA – sempre use bloco {}:

if (isAlien == false) {

    System.out.println("Não é alienígena!");

    System.out.println("Bem-vindo, humano!");

}

// Com else – executa quando condição é false:

if (isAlien == false) {

    System.out.println("Humano detectado.");

} else {

    System.out.println("Alienígena detectado!");

}
```

■ ■ **Atenção:** SEMPRE use chaves {} no if-then, mesmo com uma única instrução. Sem chaves, apenas a primeira linha é afetada pelo if — linhas adicionadas depois executarão sempre, causando bugs difíceis de detectar.

CAPÍTULO 03

= vs == e o Operador NOT !

A diferença crítica entre atribuição e igualdade

A Diferença Fundamental

Confundir `=` com `==` é um dos bugs mais comuns em programação. O operador `=` (um sinal de igual) é o operador de atribuição — define o valor de uma variável. O operador `==` (dois sinais de igual) é o operador de igualdade — compara dois valores e retorna um boolean. O IntelliJ não marca como erro usar `=` em um boolean dentro de um `if`, tornando esse bug especialmente traiçoeiro.

```
boolean isAlien = false; // = ATRIBUI false a isAlien

// == COMPARA isAlien com false:

if (isAlien == false) { System.out.println("OK"); }

// ■■ ERRADO – ATRIBUI false e testa o resultado:

// if (isAlien = false) { } // compila, mas comportamento errado!

// ■ CORRETO e IDIOMÁTICO em Java:

if (!isAlien) { System.out.println("Não é alien"); }

if (isAlien) { System.out.println("É alien"); }
```

■ **Regra:** Prefira `if (!var)` e `if (var)` para testar booleanos em vez de `if (var == false)` e `if (var == true)`. É mais conciso, mais seguro e é a forma idiomática Java.

CAPÍTULO 04

AND && e OR ||

Combinando múltiplas condições em expressões booleanas

Operador AND &&

O operador **&&** (AND lógico) retorna **true** somente quando AMBAS as condições são verdadeiras. Se qualquer uma for falsa, o resultado é false. Java usa short-circuit evaluation: se o primeiro operando já for false, o segundo nem é avaliado — eficiência automática.

Operador OR ||

O operador **||** (OR lógico) retorna **true** quando PELO MENOS UMA das condições é verdadeira. Só retorna false quando ambas são false. Short-circuit: se o primeiro operando já for true, o segundo nem é avaliado.

```
// AND (&&) – ambos devem ser true:

boolean temCarteira = true, temVeiculo = true, temGasolina = false;

if (temCarteira && temVeiculo) {

    System.out.println("Pode dirigir!"); // imprime

}

if (temVeiculo && temGasolina) {

    System.out.println("Pode sair!"); // NÃO imprime

}

// OR (||) – pelo menos um deve ser true:

boolean temDinheiro = false, temCartao = true;
```

```
if (temDinheiro || temCartao) {  
  
    System.out.println("Pode comprar!"); // imprime  
  
}
```

CAPÍTULO 05

Precedência de Operadores

Ordem de avaliação e uso de parênteses para clareza

Por que a Precedência Importa?

A precedência de operadores define qual operação é avaliada primeiro quando há múltiplos operadores em uma expressão. Em Java, a ordem simplificada é: parênteses () → operadores unários (!, ++, --) → multiplicativos (*, /, %) → aditivos (+, -) → relacionais (<, >, <=, >=) → igualdade (==, !=) → AND lógico (&&) → OR lógico (||) → ternário (?) → atribuição (=).

```
// Sem parênteses – pode surpreender:

int resultado = 2 + 3 * 4; // = 14 (não 20!) – * antes de +

// Com parênteses – garante a ordem desejada:

int resultado2 = (2 + 3) * 4; // = 20

// && tem precedência sobre ||:

boolean x = true || false && false; // true (&& antes de ||)

boolean y = (true || false) && false; // false (parêntese muda tudo)

// Desafio completo:

double a = 20.00, b = 80.00;

double soma = (a + b) * 100.00; // 10000.0

double resto = soma % 40.00; // 0.0

boolean semResto = (resto == 0.0); // true
```

```
if (!semResto) System.out.println("Tem resto!");
```

■ **Dica:** Use parênteses explícitos quando combinar && e || na mesma expressão. O compilador e seus colegas agradecerão a clareza extra.

CAPÍTULO 06

Operador Ternário ?:

O único operador de três operandos — atalho para if-else simples

O que é o Operador Ternário?

O operador ternário é o único em Java que usa três operandos e é formalmente chamado de operador condicional. É um atalho elegante para o if-then-else quando você precisa escolher entre dois valores baseado em uma condição. A sintaxe é: `variavel = (condicao) ? valorSeTrue : valorSeFalse`.

```
int idade = 20;

// if-else (forma longa):

String texto;

if (idade >= 18) { texto = "Maior de idade"; }

else { texto = "Menor de idade"; }

// Ternário (forma concisa – mesmo resultado):

String texto2 = (idade >= 18) ? "Maior de idade" : "Menor de idade";

// ■ Anti-padrão: ternário para atribuir boolean:

boolean isMaior = (idade >= 18) ? true : false; // redundante!

// ■ Forma correta – a condição JÁ É boolean:

boolean isMaior2 = (idade >= 18);
```

■ **Dica:** O ternário brilha para expressões simples de uma linha. Para lógica com múltiplas ações, prefira if-else. Ternários aninhados prejudicam muito a legibilidade.

CAPÍTULO 07

Boas Práticas e Erros Comuns

Convenções Java, atalhos IntelliJ e armadilhas para evitar

Erros Mais Comuns

- Usar `=` no lugar de `==` em condições — especialmente com boolean, o IntelliJ não avisa.
- Omitir `{ }` no if — causa bugs silenciosos quando mais linhas são adicionadas depois.
- Confundir `&&` com `||` — AND exige ambos; OR aceita qualquer um.

Todos os Operadores Vistos

```
= // Atribuição: int x = 10;

== / != // Igualdade / Diferença: x == 10 → true

! // NOT: !true → false

&& // AND lógico: true && false → false

|| // OR lógico: true || false → true

?: // Ternário: String s = (x > 5) ? "maior" : "menor";
```

Parabéns por concluir a apostila!

Com o domínio da lógica condicional e dos operadores booleanos, você tem as ferramentas para fazer seu programa tomar decisões. Próximo passo: loops e estruturas de repetição.

Complete Java Masterclass